

Six points, one line, reached two ways

First the line that least squares gives you exactly, from a formula. Then throw the formula away and reach the same line by walking downhill — the only method that will still work when the model is a network.

Scaffolded lab

20 minutes

Pure NumPy

Compare two answers

Before you start

Setup · given

1 · the exact fit

2 · the same line by descent

3 · compare the two answers

Reflection

If you finish early

Before you start

Open the provided `lab07.ipynb`. Loading the data, plotting and the checks are given; the two ideas — the exact formula and the gradient step — are yours.



By the end of this lab you can

- Compute the least-squares line on six points from its closed form.
- Reach the same line by gradient descent on two parameters at once.
- Compare the two answers numerically, and say how close is close enough.
- Explain why the course keeps the method that is slower.

LAB 7

Setup · given

Six points, the same six the lecture used. Run this and look at the scatter before anything else: you should be able to guess the slope to one significant figure.

given

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([0., 1., 2., 3., 4., 5.])
y = np.array([1.0, 2.9, 5.2, 6.8, 9.1, 11.2])

def mse(w, b):
    return np.mean((w * x + b - y) ** 2)

plt.scatter(x, y)
plt.xlabel("x"); plt.ylabel("y"); plt.title("Six points")
plt.show()
```

TASK 1 · 5 MINUTES

the exact fit

1

The least-squares line has a closed form. With $\bar{x}$ and $\bar{y}$ the means:

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$$

Write both as NumPy expressions — one line each, no loop.

your turn

PYTHON

```
w_exact = None      # TODO
b_exact = None      # TODO

print("exact: w =", round(w_exact, 4), " b =", round(b_exact, 4), " loss =",
      round(mse(w_exact, b_exact), 4))

# checked against NumPy's own least-squares fit
assert np.allclose([w_exact, b_exact], np.polyfit(x, y, 1)), "compare your formula with the
one above, symbol by symbol"
```

one version that works

PYTHON

```
xm, ym = x.mean(), y.mean()
w_exact = np.sum((x - xm) * (y - ym)) / np.sum((x - xm) ** 2)
b_exact = ym - w_exact * xm

print("exact: w =", round(w_exact, 4), " b =", round(b_exact, 4), " loss =",
      round(mse(w_exact, b_exact), 4))
assert np.allclose([w_exact, b_exact], np.polyfit(x, y, 1))
```

what it should print

OUTPUT

```
exact: w = 2.0343 b = 0.9476 loss = 0.0188
```



The loss at the exact answer is **not zero**. Six real measurements do not lie on any line — which is exactly what Lab 6's task 6 warned you about.

TASK 2 · 7 MINUTES · THE MAIN TASK

the same line by descent

2

Now forget the formula. Start from `w = b = 0` and walk downhill on the loss. Two parameters move at once, but each step is Lab 6's step: *value minus learning rate times slope*.

The slopes of the mean squared error are

$$\frac{\partial L}{\partial w} = 2 \cdot \text{mean}((wx + b - y) \cdot x), \quad \frac{\partial L}{\partial b} = 2 \cdot \text{mean}(wx + b - y)$$



 **Update both from the same error.** Compute `error` once, then both gradients, then both updates. Updating `w` and then recomputing the error for `b` is a different algorithm, and a subtle bug.

your turn

PYTHON

```
w, b = 0.0, 0.0
lr = 0.02
losses = []

for epoch in range(1000):
    error = w * x + b - y
    losses.append(np.mean(error ** 2))
    dw = None # TODO
    db = None # TODO
    w = w - lr * dw
    b = b - lr * db

print("descent: w =", round(w, 4), " b =", round(b, 4), " loss =", round(losses[-1], 4))
assert abs(losses[-1] - mse(w_exact, b_exact)) < 1e-3, "descent should land within 0.001 of the exact loss"
```

one version that works

PYTHON

```
w, b = 0.0, 0.0
lr = 0.02
losses = []
for epoch in range(1000):
    error = w * x + b - y
    losses.append(np.mean(error ** 2))
    dw = 2 * np.mean(error * x)
    db = 2 * np.mean(error)
    w = w - lr * dw
    b = b - lr * db
print("descent: w =", round(w, 4), " b =", round(b, 4), " loss =", round(losses[-1], 4))
assert abs(losses[-1] - mse(w_exact, b_exact)) < 1e-3
```

what it should print

OUTPUT

```
descent: w = 2.0343  b = 0.9476  loss = 0.0188
```

TASK 3 · 5 MINUTES

compare the two answers

3

The plotting is given. Your job is the number underneath it: **after how many epochs** was the descended loss within 1% of the exact loss?

your turn

PYTHON

```
fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
xs = np.linspace(0, 5, 50)
ax[0].scatter(x, y, label="data")
ax[0].plot(xs, w_exact * xs + b_exact, label="exact")
ax[0].plot(xs, w * xs + b, "--", label="descent")
ax[0].set_xlabel("x"); ax[0].set_ylabel("y"); ax[0].legend(); ax[0].set_title("Two lines")
ax[1].plot(losses, label="descent")
ax[1].axhline(mse(w_exact, b_exact), color="red", linestyle=":", label="exact loss")
ax[1].set_yscale("log"); ax[1].set_xlabel("epoch"); ax[1].set_ylabel("MSE"); ax[1].legend();
ax[1].set_title("Loss")
plt.show()

epochs_to_1pct = None # TODO: the first epoch whose loss is within 1% of the exact loss
print("within 1% after", epochs_to_1pct, "epochs")
```

one version that works

PYTHON

```
target = 1.01 * mse(w_exact, b_exact)
epochs_to_1pct = next(i for i, L in enumerate(losses) if L ≤ target)
print("within 1% after", epochs_to_1pct, "epochs")
```

what it should print

OUTPUT

within 1% after 224 epochs

Reflection



Write it in the notebook, in your own words

The exact formula took one line and no loop. Descent took a thousand steps and a learning rate. In two or three sentences: why does the course keep the slow method? *Hint*: try to write the closed form for session 11's neuron, where the prediction goes through a sigmoid.

Your answer:

If you finish early

A. Rerun Task 2 with `lr = 0.1`. What happens, and why is the safe learning rate smaller here than Lab 6's cliff at 1.0? (*The size of the x values now multiplies the slope.*)

B. Centre the inputs — use `x - x.mean()` in place of `x` — and rerun at `lr = 0.1`. Say in one sentence what centring did for the walk.

your turn

PYTHON

```
# TODO (optional extensions A, B)
```